

Refatoração de Testes e Detecção de Test Smells

Disciplina:	Engenharia de Software / Qualidade de Software
Aluno:	Vitor Rebula Nogueira
Matrícula:	820520
Data:	15 de Maio de 2026

1. Análise de Test Smells

A suíte de testes original (*userService.smelly.test.js*) foi analisada manualmente e com o auxílio do ESLint. Foram identificados três Test Smells clássicos, descritos a seguir:

1.1 Lógica Condicional no Teste (Conditional Logic)

O teste *'deve desativar usuários se eles não forem administradores'* utiliza um laço **for** e um bloco **if/else** internamente para verificar diferentes comportamentos dependendo do tipo de usuário:

```
for (const user of todosOsUsuarios) { const resultado =  
  userService.deactivateUser(user.id); if (!user.isAdmin) {  
    expect(resultado).toBe(true); // só roda para usuário comum } else {  
    expect(resultado).toBe(false); // só roda para admin } }
```

Por que é um bad smell? Quando um teste contém lógica condicional, torna-se difícil entender o que exatamente está sendo verificado. Cada ramificação do *if/else* representa um cenário diferente; misturá-los num único teste obscurece a intenção e torna a falha difícil de diagnosticar. O ESLint detectou esse padrão com a regra **jest/no-conditional-expect** (4 erros nas linhas 44, 46, 49 e 73).

Risco: Se o teste falhar, não fica claro qual dos cenários é o responsável. Além disso, dependendo dos dados de entrada, alguns ramos do *if* podem nunca ser executados, criando uma falsa sensação de cobertura.

1.2 Teste Ansioso / Eager Test

O teste *'deve criar e buscar um usuário corretamente'* verifica duas responsabilidades distintas numa mesma função: a criação do usuário e a busca pelo ID. Ele executa dois *Acts* seguidos e múltiplos *Asserts*:

```
// Act 1: Criar const usuarioCriado = userService.createUser(...);
expect(usuarioCriado.id).toBeDefined(); // Act 2: Buscar const usuarioBuscado =
userService.getUserById(usuarioCriado.id);
expect(usuarioBuscado.nome).toBe(dadosUsuarioPadrao.nome);
expect(usuarioBuscado.status).toBe('ativo');
```

Por que é um bad smell? O princípio de um bom teste unitário é verificar uma única coisa por vez. Quando um teste valida múltiplos comportamentos, viola o princípio da responsabilidade única. Se falhar, o desenvolvedor precisará analisar todo o bloco para identificar qual das asserções quebrou.

Risco: Uma falha na primeira asserção interrompe o teste inteiro, escondendo informações sobre as asserções seguintes. A manutenção também se torna mais custosa, pois qualquer alteração em *createUser* ou *getUserById* pode quebrar o mesmo teste.

1.3 Teste Ignorado / Skipped Test

O último teste da suíte utiliza **test.skip** com um corpo vazio e um comentário *TODO*, indicando que o comportamento nunca foi de fato implementado:

```
test.skip('deve retornar uma lista vazia quando não há usuários', () => { //
  TODO: Implementar este teste depois. });
```

Por que é um bad smell? Testes ignorados são silenciosos. Eles não falham, não executam e não protegem funcionalidade alguma. Com o tempo, acumulam-se na base de código e são esquecidos. O ESLint reportou esse problema com dois avisos: **jest/no-disabled-tests** (testes não devem ser pulados) e **jest/expect-expect** (teste sem nenhuma asserção).

Risco: A funcionalidade de lista vazia pode nunca ser coberta por testes, e um bug nesse comportamento passará despercebido indefinidamente.

2. Processo de Refatoração

O teste escolhido para demonstrar a refatoração é o mais problemático da suíte original: *'deve desativar usuários se eles não forem administradores'*. Ele concentra dois smells ao mesmo tempo: Lógica Condicional e Eager Test.

Antes — Código Problemático

```
test('deve desativar usuários se eles não forem administradores', () => { const
  usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30); const
  usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin]; for (const user of
  todosOsUsuarios) { const resultado = userService.deactivateUser(user.id); if
  (!user.isAdmin) { expect(resultado).toBe(true); const usuarioAtualizado =
  userService.getUserById(user.id);
  expect(usuarioAtualizado.status).toBe('inativo'); } else {
  expect(resultado).toBe(false); } } });
```

Depois — Código Limpo (padrão AAA)

```
test('deve retornar true ao desativar um usuário comum', () => { // Arrange const
  usuario = userService.createUser('Comum', 'comum@teste.com', 30); // Act const
  resultado = userService.deactivateUser(usuario.id); // Assert
  expect(resultado).toBe(true); }); test('deve alterar o status do usuário comum
  para "inativo"', () => { // Arrange const usuario =
  userService.createUser('Comum', 'comum@teste.com', 30); // Act
  userService.deactivateUser(usuario.id); const atualizado =
  userService.getUserById(usuario.id); // Assert
  expect(atualizado.status).toBe('inativo'); }); test('deve retornar false ao
  tentar desativar um administrador', () => { // Arrange const admin =
  userService.createUser('Admin', 'admin@teste.com', 40, true); // Act const
  resultado = userService.deactivateUser(admin.id); // Assert
  expect(resultado).toBe(false); });
```

Decisões de Refatoração

- 1. Eliminação do laço e do if/else:** Os dois cenários (usuário comum e administrador) foram separados em testes independentes. Cada teste verifica exatamente um comportamento, com nome descritivo que documenta a intenção.
- 2. Aplicação rigorosa do padrão AAA:** Cada teste passou a ter três blocos claramente delimitados — Arrange (preparação dos dados), Act (execução da ação) e Assert (verificação do resultado). Isso torna o teste autoexplicativo.
- 3. Verificação de comportamento, não de implementação:** O relatório foi testado com **toContain** em vez de verificar a string exata formatada. Assim, mudanças menores no formato do relatório não quebram os testes desnecessariamente.

4. Cobertura do teste ignorado: O teste *skipped* foi implementado de fato, verificando que *generateUserReport* retorna a mensagem correta quando não há usuários cadastrados.

3. Relatório da Ferramenta (ESLint)

3.1 Primeira execução — arquivo smelly

Após configurar o `.eslintrc.json` com o plugin `eslint-plugin-jest`, o comando `npx eslint .` retornou os seguintes problemas no arquivo original:

```
/home/levty/test-smelly/test/userService.smelly.test.js 44:9 error Avoid calling
`expect` conditionally jest/no-conditional-expect 46:9 error Avoid calling
`expect` conditionally jest/no-conditional-expect 49:9 error Avoid calling
`expect` conditionally jest/no-conditional-expect 73:7 error Avoid calling
`expect` conditionally jest/no-conditional-expect 77:3 warning Tests should not
be skipped jest/no-disabled-tests 77:3 warning Test has no assertions
jest/expect-expect ✖ 6 problems (4 errors, 2 warnings)
```

O linter identificou automaticamente os mesmos problemas encontrados na análise manual: quatro usos de **expect** dentro de blocos condicionais (linhas 44, 46, 49 e 73) e o teste ignorado sem asserções (linha 77). A ferramenta aponta o número exato da linha, o código da regra violada e uma descrição objetiva do problema.

3.2 Segunda execução — arquivo clean

Após a refatoração completa no arquivo `userService.clean.test.js`, o ESLint foi executado novamente. O resultado foi:

```
$ npx eslint . -> nenhum problema reportado no arquivo clean ✖ 6 problems (4
errors, 2 warnings, somente no arquivo smelly original)
```

O arquivo refatorado passou pela análise estática sem nenhum aviso ou erro, confirmando que todos os smells foram corretamente eliminados.

3.3 Como a ferramenta automatizou a detecção

A análise manual exige que o desenvolvedor leia todo o arquivo atentamente e conheça os padrões problemáticos. Já o ESLint executa em milissegundos e não depende de atenção humana. As regras configuradas (*jest/no-conditional-expect*,

jest/no-disabled-tests, *jest/expect-expect*) codificam o conhecimento sobre boas práticas de teste e o aplicam de forma consistente em toda a base de código, a cada execução — inclusive em pipelines de CI/CD, bloqueando a integração de código problemático antes mesmo do merge.

4. Conclusão

Este trabalho demonstrou de forma prática como Test Smells afetam a qualidade e a sustentabilidade de uma suíte de testes — mesmo quando todos os testes estão passando. A suíte original atingia 100% de cobertura aparente, mas continha testes frágeis, com lógica oculta e cenários ignorados, que nunca seriam capazes de detectar regressões sutis.

A aplicação do padrão **Arrange, Act, Assert** trouxe clareza estrutural aos testes: cada bloco tem um propósito definido, e o leitor entende imediatamente o que está sendo preparado, executado e verificado. A separação de testes ansiosos em testes menores e focados também permitiu diagnosticar falhas com muito mais precisão.

A análise estática com ESLint e o plugin **eslint-plugin-jest** provou seu valor ao detectar automaticamente os mesmos problemas identificados manualmente, apontando linhas exatas e regras violadas. Integrada a um pipeline de CI/CD, essa ferramenta atua como uma rede de segurança contínua, impedindo que novos smells sejam introduzidos na base de código.

Em conjunto, a prática de escrever testes limpos e a utilização de ferramentas de análise estática formam uma base sólida para projetos de software sustentáveis: testes que realmente protegem o sistema, são fáceis de manter e evoluem junto com a aplicação ao longo do tempo.